

AI FAQ ASSISTANT

Description

The AI FAQ Assistant is a robust, secure, and intelligent REST API platform designed to streamline FAQ creation, user authentication, and AI-powered content management. Built with Node.js and Express.js, the application utilizes MongoDB as its primary data store along with Mongoose ODM for elegant data modeling and schema validation.

Security is central to the architecture, implementing JWT (JSON Web Tokens) for stateless user sessions and route guarding alongside bcrypt for strong password encryption. The backend handles intensive operations like automated FAQ generation, semantic search queries, and dynamic context parsing. It also features robust centralized error handling and response status sanitization to prevent unhandled database leakage and ensure operational durability.

Scenario-Based Case Study

Background

Priya is a customer support operations manager who handles massive backlogs of repetitive user inquiries but struggles to organize entries, classify topics, write precise summaries, and track client question trends efficiently.

Problem

- Difficulty generating and maintaining up-to-date documentation workflows.
- Limited visibility into semantic search match keywords and rapid query resolutions.
- Limited visibility into semantic search match keywords and rapid query resolutions.
- Time-consuming manual processes for drafting complex support answers.
- Lack of unified database permission layers protecting private documentation from public adjustments.

Team id: SWTID-2026-3666

Name	Email Id	NM Id
Subi Manoj s (leader)	Saiselvin097@ gmail.com	CE451ADEF4BB28D98B34A67DA9926902
Selvakumar e (Member)	Selvakumar.e14 30@gmail.com	78DAC8F68442FCB8BC646E124CF8AF62

Sham p (Member)	Shamparthiban143@gmail.com	AC99CD91CCAFD9B86854B2F16F0C9A88
Silambarasan k (Member)	Simbuk490@gmail.com	2396E0F166304F594BF47052470E53F7

PHASE-1

Software Requirements

- **Operating System: Windows 10/11, macOS, or Linux (supports cross-platform operations).**
- **Node.js (v18 or above): Runtime ecosystem running server logic and routing infrastructure.**
- **npm (v9 or above): Node package manager to control operational dependencies.**
- **Express.js (v4/v5 Ecosystem): Lightweight routing web framework to construct backend RESTful entry points.**
- **MongoDB / Mongoose ODM: Document NoSQL engine storing application users and curated data entities.**
- **Postman / ThunderClient: API verification toolkit to assert schema validation responses across administrative route shields.**
- **Code Editor: Visual Studio Code or similar IDE environment.**

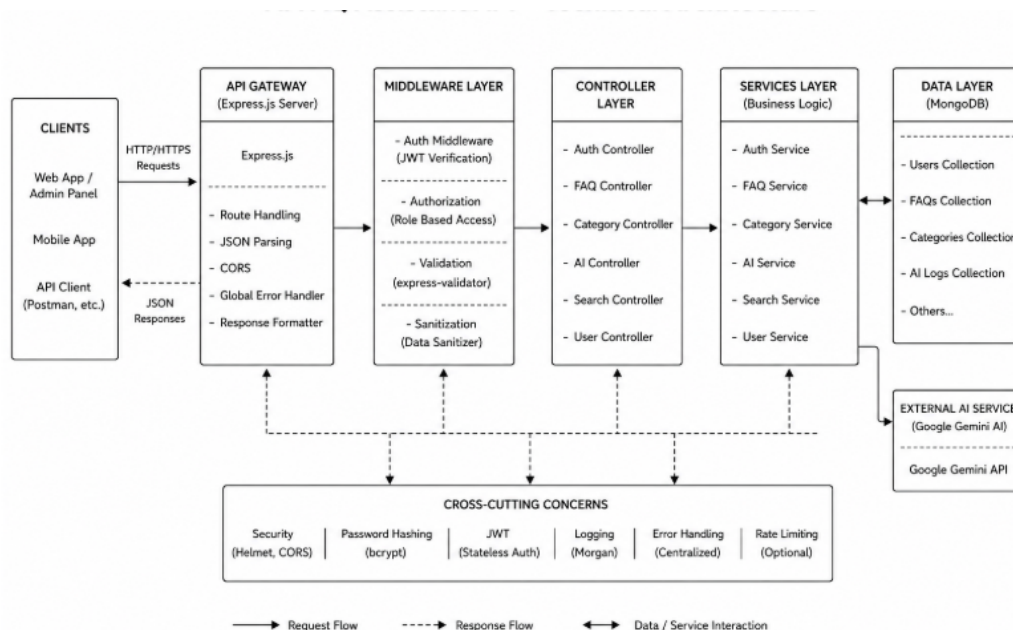
PHASE-2

Hardware

Requirements

- **Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.**
- **RAM: 8 GB minimum (16 GB recommended for concurrent instances of MongoDB processing layers).**
- **Storage: 1 GB of available disk workspace.**

Epic-1: Project Architecture



The AI FAQ Assistant application exposes structured endpoints handled by a layered server architecture model.

Incoming requests undergo filter processing before interacting with data services.

Components Definition

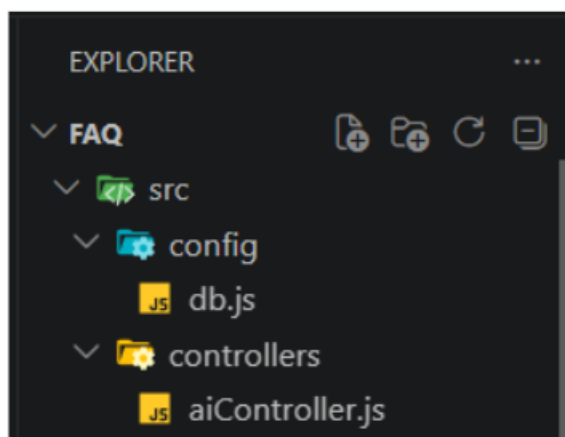
- **Interface: API tools like Postman query distinct route paths to extract JSON entities.**
- **Express Server Gateway: Monitors network ports, maps explicit HTTP methods (GET, POST, PUT, DELETE), parses incoming JSON payloads, and handles CORS headers.**
- **Authentication Middleware: Extracts tokens from headers, validates user context signatures, and evaluates authorization thresholds for guarded resource paths.**
- **Granular Services (Logic Modules): Enforces isolated domain rules for user session life cycles and operational query modifications.**
- **Database Interface Layer: Mongoose translates operational application logic into performant native MongoDB queries.**
- **AI Flow (Service Integration): Connected to external AI models via Google Gemini SDK integrations, letting the controller layer offload complex question-answering tasks and automated topic generation workflows.**

PHASE-3

**Project Setup and
Configuration**

Creating Project Folder

1. Create a new folder with your <project name>.
2. Inside that folder, create a new subfolder named src.
3. Now open that folder in VS Code.



Server Setup (npm init)

Open your root folder terminal in VS Code and run:

Bash

```
npm init -y
npm install express mongoose dotenv cors jsonwebtoken bcrypt
@google/genai
```

File and Folder Structure Matrix

Plaintext

Epic 3: Backend Development

Core System Script Implementations

1. Gateway Server Node (src/server.js)

This main entrance mechanism aggregates environment variables, triggers database listeners, and binds network listeners to active environment ports:

Phase-4

Database Configuration

Mongoose Schema Definitions

1. Identity Resource Document Schema (src/models/User.js)

Enforces profile field criteria validations and hooks password hashing steps prior to user document persistence:

```
src > models > User.js > ...
1  const mongoose = require('mongoose');
2  const bcrypt = require('bcrypt');
3
4  const userSchema = new mongoose.Schema(
5  {
6    name: {
7      type: String,
8      required: [true, 'Name is required'],
9      trim: true,
10   },
11   email: {
12     type: String,
13     required: [true, 'Email is required'],
14     unique: true,
15     lowercase: true,
16     trim: true,
17     match: [/^\s*@\s*\.\s*$/, 'Please provide a valid email address'],
18   },
19   password: {
20     type: String,
21     required: [true, 'Password is required'],
22     minlength: [6, 'Password must be at least 6 characters long'],
23   },
24 },
25 {
26   timestamps: true,
27 }
28 );
29
30 // Pre-save hook to hash the password before saving
31 userSchema.pre('save', async function () {
32   if (!this.isModified('password')) {
33     return;
34   }
35   const salt = await bcrypt.genSalt(10);
36   this.password = await bcrypt.hash(this.password, salt);
37 });
```

2. FAQ Resource Document Schema (src/models/FAQ.js)

Models core properties and maps a distinct relational reference link back to the source creator document:

```
src > models > FAQ.js > ...
1  const mongoose = require('mongoose');
2
3  const faqSchema = new mongoose.Schema(
4  {
5    question: {
6      type: String,
7      required: [true, 'Question is required'],
8      trim: true,
9    },
10   answer: {
11     type: String,
12     required: [true, 'Answer is required'],
13     trim: true,
14   },
15   category: {
16     type: String,
17     required: [true, 'Category is required'],
18     enum: {
19       values: ['Technology', 'Education', 'Health', 'Banking', 'General'],
20       message: '[VALUE] is not a valid category. Choose from Technology, Education, Health, Banking, General',
21     },
22     default: 'General',
23   },
24   createdBy: {
25     type: mongoose.Schema.Types.ObjectId,
26     ref: 'User',
27     required: [true, 'Creator user reference is required'],
28   },
29 },
30 {
31   timestamps: true,
32 }
33 );
34
35 const FAQ = mongoose.model('FAQ', faqSchema);
36
```

PHASE-5

Project Executions

Running the API Application

- **Install Postman or Thunder Client (VS Code extension).**

- Start the backend development server environment parameters:

Bash

npm run dev

API Testing Specification Demo

To verify operational structural integrity via execution endpoints, map endpoints using the configurations below:

1. Identity Onboarding Path Registration

- Route: POST /api/auth/register
- Access: Public
- Payload Request Body JSON:

```
{  
  "name": "sk",  
  "email": "sk@gmail.com",  
  "password": "123456789"  
}
```

PHASE-6

API Testing

Specification Demo

To verify operational structural integrity via execution endpoints, map endpoints using the configurations below:

1. Identity Onboarding Path Registration

- **Route: POST /api/auth/register**

- Access: Public
- Payload Request Body JSON:

1.http://localhost:5000/api/auth/register

```
{  
  "name":"Sk",  
  "email":sk@gmail.com,  
  "password":"123456789"  
}
```

2.http://localhost:5000/api/auth/login

```
{  
  "email":sk@gmail.com,  
  "password":"123456789"  
}
```

PHASE-7

Resource Entity Creation (FAQ)

Route: POST /api/faqs

Access: Private (Bearer <JWT Token>)

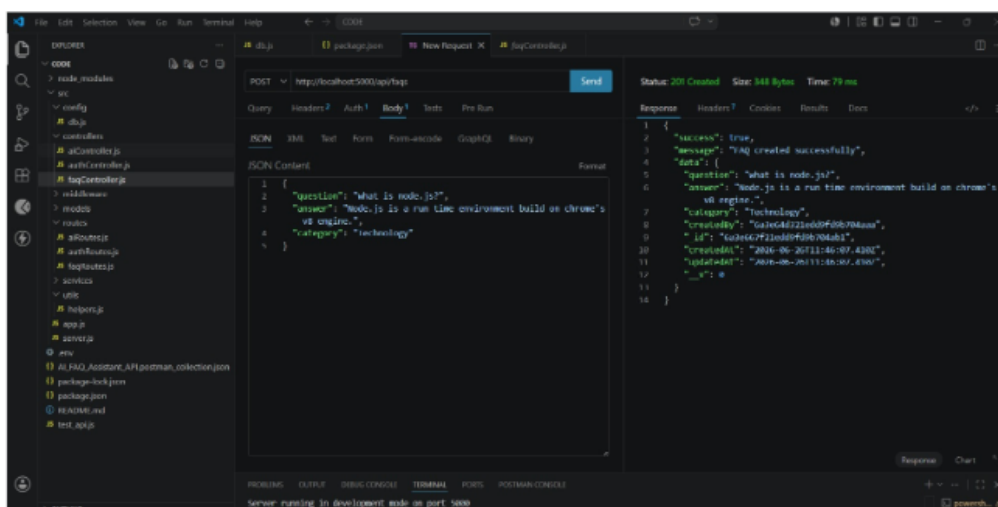
Payload Request Body JSON:

```
{  
  "question": "How do I configure custom category  
tags?",
```

```

    "answer": "Navigate to settings panel, append
keyword      tags arrays, and save schema.",
    "category": "Configuration"
}

```



PHASE-8

AI Intelligent Content Automation (Generate FAQ Pair)

AI Intelligent Content Automation (Generate FAQ Pair)

- Route: POST /api/ai/generate-faq
- Access: Private (Bearer <JWT Token>)
- Payload Request Body JSON:

{

```
"topic": "Mongoose schema indexing validation runtime  
workflow optimization"  
}
```

DEMO VIDEO:

API TESTING VIDEO: